

05 什么是动力学模型？

作者 NILUSHANAN KULASINGHAM

阅读约 15 分钟 · 可交互

一个在训练时每一步都拿到真相的模型，一上线拿到的就是它自己上一次的答案。是什么把过去带着往前走，以及为什么那个头条准确率数字量的是另一份工作。

本章的图都是可交互的。请到 worldmodels101.com/zh/chapters/dynamics 在线按一按、拖一拖。

如果你读过上一章，你手里已经有了一个状态：一小串数字，代表摄像机看到的东西。接下来这一步，是让这个状态动起来。动力学模型，也叫转移模型，只做一件事：拿走你知道的，拿走你做了的，产出你接下来知道的。

别扭的是「知道」这个词，因为它盖住的东西比上一张画面多。它是曾经发生过、而且现在仍然要紧的一切，被折进一个小到能随身带的东西里。墙后面那个球还在动。你四个房间之前打开的那扇门还开着。

一篇讲动力学的论文会把单步误差放在最前面，而这也是最容易让人印象深刻的数字。我被它唬住过好一阵子，直到我去看它到底量的是什麼。我们先从这件事本身说起。

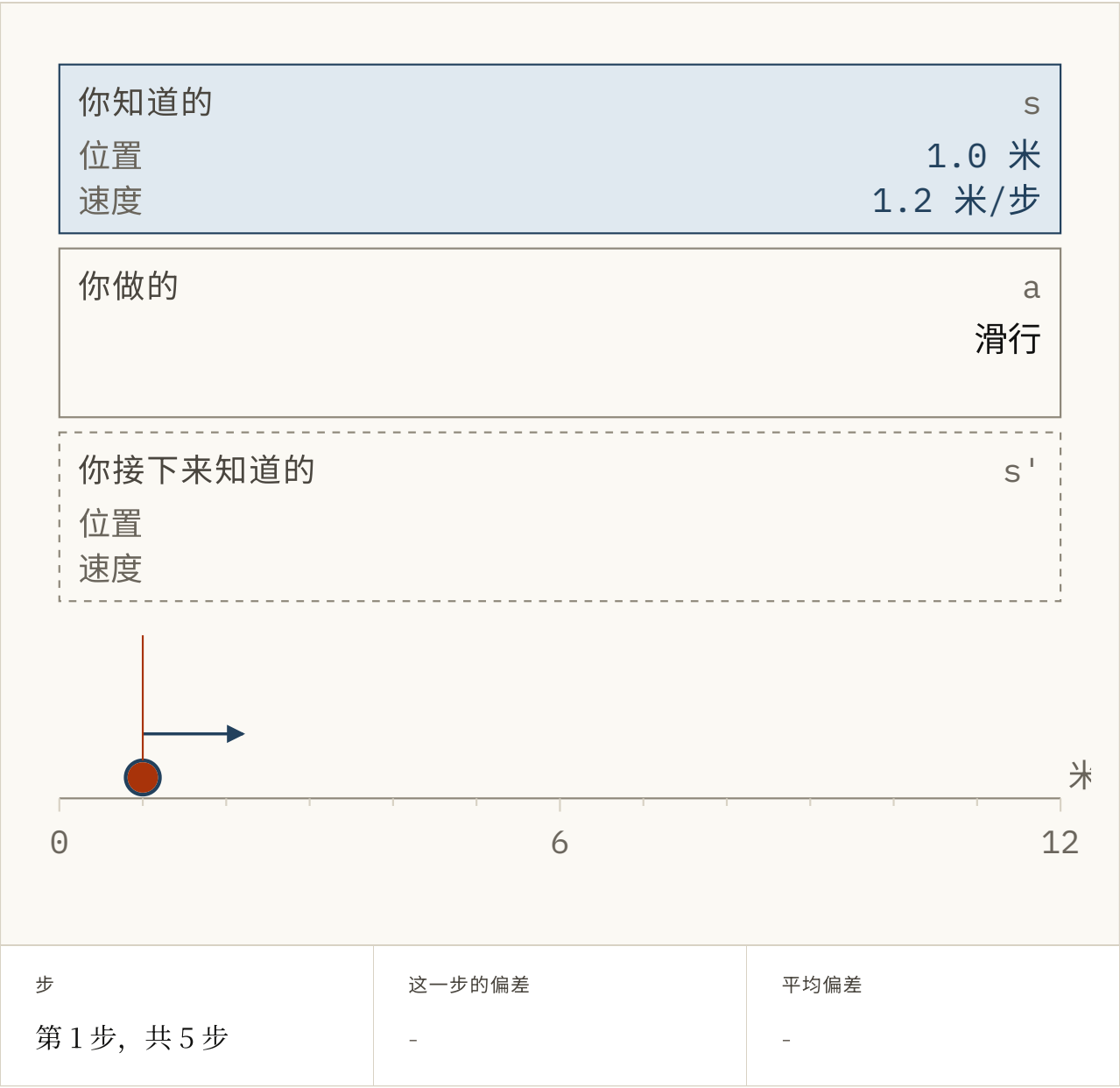


FIG. 5.1 这就是那一件事，而这一小会儿由你来当模型。状态和动作都交到你手上了，把你的答案拖到你认为球走一步之后会在的位置，然后按「核对」。这么做几次，盯住下面那个平均误差是怎么攒起来的，因为一篇讲动力学的论文，打头的就是这个平均值。而这里的每一步都是从真相出发的，这才是要记住的地方。单位仅作示意。

再说那个数字。把东西现在的位置给模型看，问它接下来会怎样。把它的答案和实际发生的比一比，这么做一万次，读出平均值。结果会很小，而且它量的确实是真东西。

它同时也在量一份模型永远不会被要求做的工作。你打算问它一百步。而从第二步开始，交到它手里的就不再是真相，而是它自己上一次的答案。

我把这个叫做第二步问题。这个领域在 2015 年给它取了名字，一年之内试过两种互相竞争的解法，此后一直没解决。我们先看看它会对一个小模型做什么。



FIG. 5.2 同一个模型，同一个偏差，两种跑法。灰线在每一步都被纠正，就像训练时那样，基本上消失在真相下面了。朱红色（橙红色）那条是同一个模型，被放开去吃自己的输出，也就是它将来实际会被使用的方式。把偏差往上推，看哪条线有反应。距离仅作参考。

从读数上看，在每步 4% 误差的情况下，被纠正的那条线走了三十步之后仍然紧贴着真相。自由运行的那条，同一个模型、同一个偏差，最后差出去三十九倍。把这个画面记住。

继承下来的习惯

这个习惯是跟着最早那批在序列上训练的网络一起来的，比世界模型早了几十年。训练时，序列模型在每一步都被展示真实的上一个值。记录里装的就是这个，而且这样训练更稳。这个领域把它叫做**教师强制**。

代价是，上线之后没有人会来给你做教师强制。真正使用的那一刻，模型拿到的是它自己上一次的答案，那个答案有点偏。而它一次都没有被要求过从「有点偏」里恢复过来。

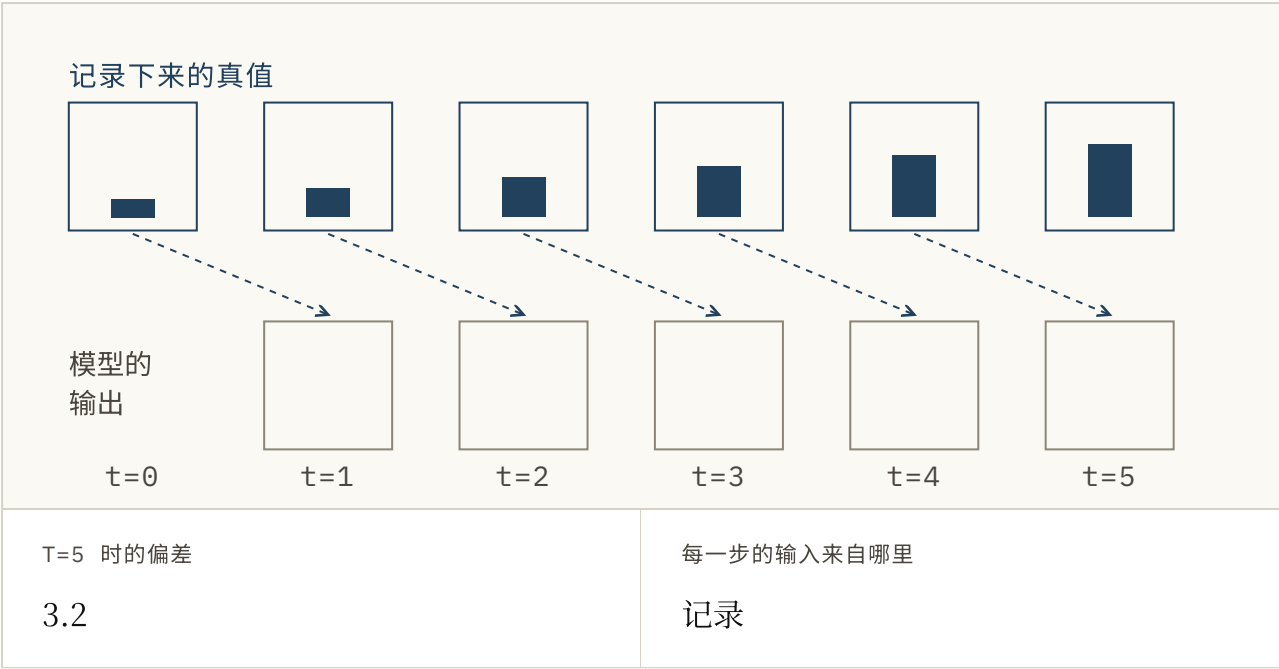


FIG. 5.3 同样五步，把喂给每一步的箭头也画了出来。把开关留在「训练」上，每一步拿到的都是记录下来真相，于是错误哪儿也去不了。切到「使用中」，每一步拿到的都是它前面那一步，于是第一个小错误会一路搭车进入后面每一步。按一下「走一步」，看看朱红色是从哪儿来的。

Samy Bengio 和他的合作者在 2015 年给这道缺口取了名字，论文是 *Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks*。Bengio 当时在 Google，做的是给照片配文字说明的网络：一次生成一个词，每个词又被喂回去当成下一个输入。这就是一次推演，而那些说明文字漂移的方式，和朱红色那条线一模一样。一篇把灰线放在最前面的论文，描述的是错的那种跑法。

说这个模型不擅长长时程，是读错了它。它是在一个输入分布上训练出来的，而你一开始往前推演，它就再也见不到那个分布了，训练循环里也没有任何东西注意到这一点。

记忆之争，1997 到 2023

我们回到「知道」这个词，因为它底下的问题是：过去到底怎么被带着走。从 1997 年起，这个问题有两个答案，两个轮流当红。

带一份摘要。 保留一块固定大小的数字。每次有新东西来，就把它折进去，然后把旧的那块扔掉。不管序列变多长，每一步的工作量都不变。

留住一个窗口。 把上下文装得下的步骤存起来。新的一步到来时，回头看它们，再决定什么要紧。这就是注意力 (对每个新步骤，把早先的步骤按有多要紧打分，再主要从分高的那些里读)。

	带一份摘要	留住上下文窗口
往前带的数字个数	256	32,768
下一步到来时，模型手里握着的东西。		
多走一步的计算量	65,536	32,768
把一个新观测折进去所需的乘加次数。		
它还看得到第一步吗？	只有摘要留下了才行	还在上下文里时，可以看到
固定大小的摘要丢掉的东西，后面补不回来。		
两列用的状态都是 256 个数字宽，所以唯一在变的只有长度。		

FIG. 5.4 两列用的状态宽度相同，所以唯一在变的只有序列有多长。把步数往上拖，盯住第二行。两列都不是赢家：对短序列来说，「全都看一遍」更便宜；对长序列来说则更贵。

摘要就是循环网络 (带环的网络：每一步的输出会被交回去，成为下一步的输入) 在做的事。摘要这一派的奠基论文是 *Long Short-Term Memory*，1997 年由 Sepp Hochreiter 与 Jürgen Schmidhuber 发表。他们的 LSTM 是一个专门造出来的环，为的是把一个事实记得比训练压力愿意让它记的更久。后来在 transformer 出现之前，语音识别和机器翻译一直是它在跑。

窗口则是 transformer 在做的事，而 transformer 就是今天各种语言模型底下的那套架构。它们从 2017 年开始这么做，那一年 Ashish Vaswani 和他在 Google 的同事发表了 *Attention Is All You Need*。标题说的就是字面意思：环被拿掉了。几年之内，窗口几乎在所有地方都取代了摘要。

然后摘要又回来了。状态空间模型保留了固定摘要，再给它换上更好的机件。S4 出自 Albert Gu 和他在斯坦福的同事，2021 年，论文是 *Efficiently Modeling Long Sequences with Structured State Spaces*。Mamba 出自 Gu 与 Tri Dao，2023 年，论文是 *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*。

这场取舍里被拿去跑基准的那一半是速度，而那是没意思的一半。一份固定摘要必须在每一步决定什么值得留下，而且当时并不知道后面会用到什么。Mamba 让了一小步：它的摘要会根据自己正在看的东西来决定留什么。注意力则把这个决定推迟到读取的时候，代价是要存着一批数量有限的早先状态。



FIG. 5.5 六个事实一个接一个地来，而一份只放得下三个的摘要必须在它们到达的当下决定留还是丢。最后来了一个问题。挑一个问题，按一下「播放」，盯住那份摘要：它没法预先看到问题，所以有时候它丢掉的正好就是要用的那个事实。切到窗口，同一个问题靠回头看就答得上来，代价是六个全都得留着。数字仅作示意。

这就是第 4 章里摄像机与决定之间那道收窄口，只是高了一层。总得有东西被扔掉，而扔它的那个东西并不知道未来。这就是为什么二十六年的对垒到现在也没分出胜负。

折中的办法

那些能用的系统共享同一个技巧。它在 2018 年随 PlaNet 一起出现，作者是 Danijar Hafner 和他的同事，论文是 *Learning Latent Dynamics for Planning from Pixels*。它从原始像素里学出一个模型，然后在模型里面做规划，而不是在像素上。

这个技巧就是往前带两样东西，而不是一样。一部分是确定的：每次都同样的方式算出来，装的是从上一个状态可靠地推出来的东西。另一部分是随机的：采样出来的，装的是仍然可能走向任一边的东西。

球的运动放进前者。门会不会开放进后者。只留确定的那部分，模型就没办法说自己没把握，于是它会自信地编出一个未来。只留随机的那部分，它就很难把东西记很久，因为每一步都在往里加噪声。

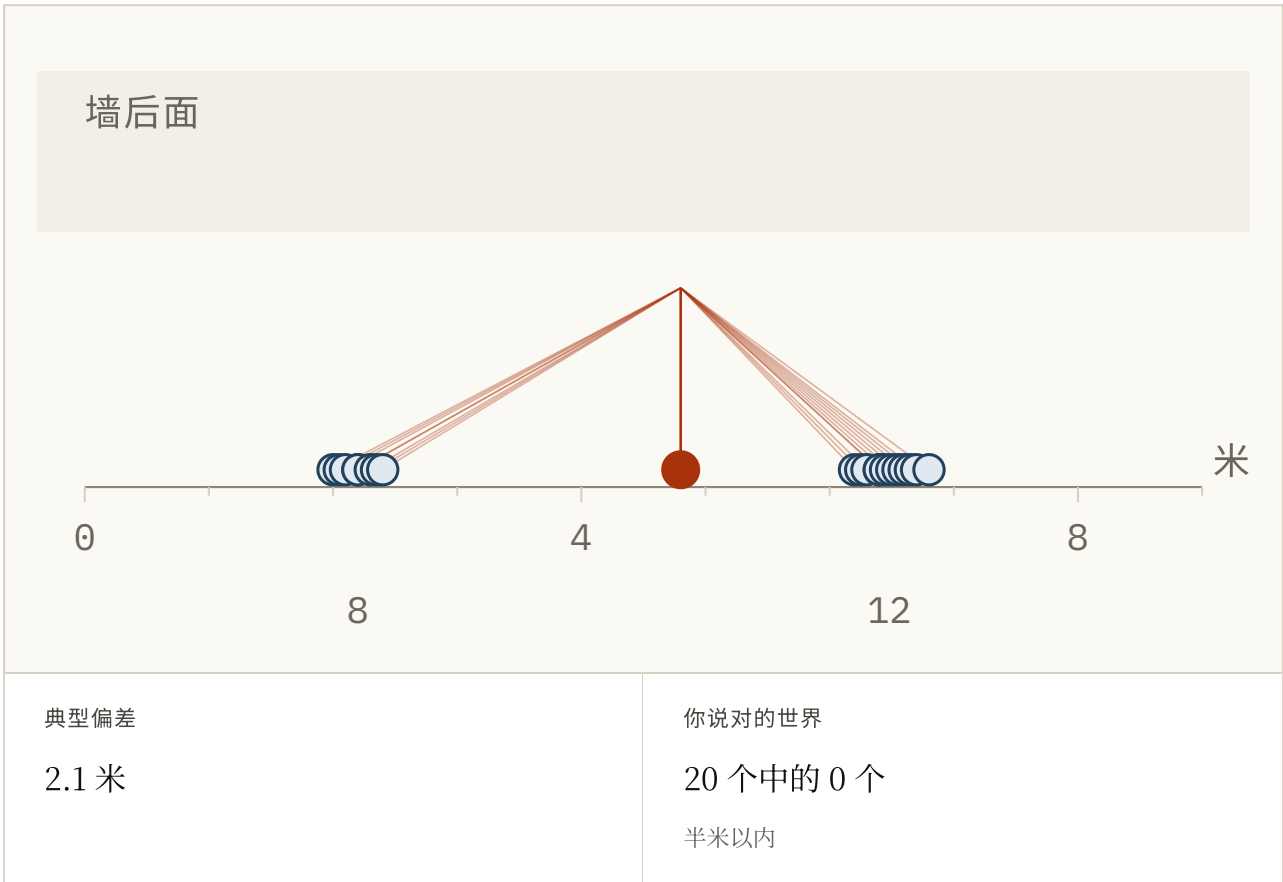


FIG. 5.6 球已经走到墙后面去了，它要么撞上路缘停住了，要么就一直滚了过去。把那个「唯一的答案」沿着地面拖，看两个读数怎么往两边扯：典型误差最小的位置落在两团中间，而球从来不会真的在那儿。然后切到「交回一片散布」，看看状态的后一半买到了什么。二十个世界，仅作示意。

PlaNet 把这个对比跑了一遍，主张两个都带，理由是单独留下任何一个，失败的方向都不一样。一年之后是 Dreamer：同一批人发表了 *Dream to Control*，让一个智能体在模型想象出来的推演里学会怎么行动。

这个拆分状态进了 Dreamer，也进了此后每一代 Dreamer，一直到 2025 年那篇 *Nature* 论文。

与第二步问题的四种停战

没有人消除掉第二步问题，我也不指望有谁能做到。这个领域手里有的，是四种「不被它毁掉」的办法，我们一个一个来看。

让它在自己的输出上训练。如果抱怨的是模型从来没练过从自己的错误里恢复，那就让它在训练时犯一些。有一部分时间喂它自己的预测，并且随着它变好把这个比例往上调。这就是 Bengio 的 *计划采样*，四种里最直接的一种。

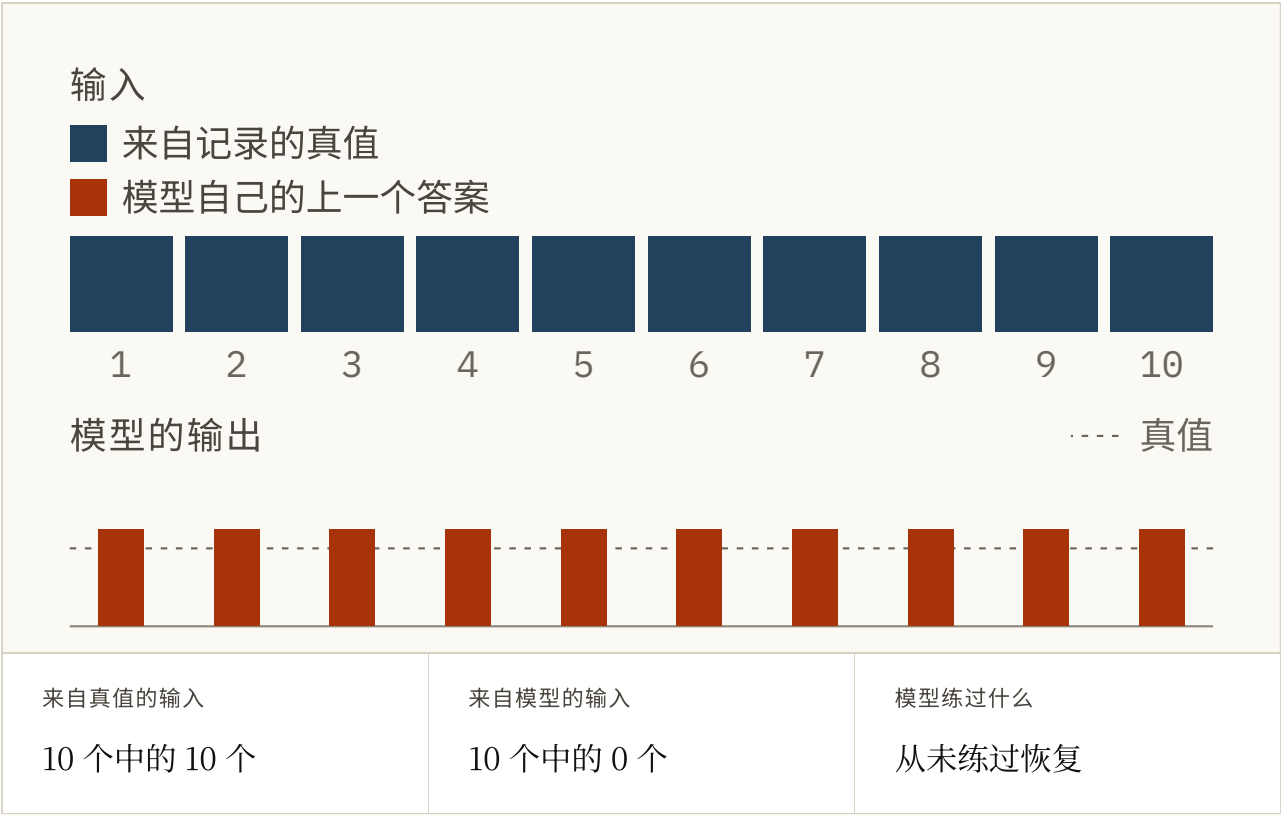


FIG. 5.7 连着十个训练步。滑块控制的是模型有多经常拿到自己上一次的答案，而不是录下来的真相。拖到零就是教师强制，每一个输入都是真相。往上拖，朱红色的输入就出现了，那是模型在自己的错误上练习。按一下「运行计划」，像 Bengio 那样让这个比例在一整轮里逐步升上去，再读下面的结论。数字仅作参考。

训练整段推演，而不是单步。别再给单步准确率打分，改成拿一整段想象出来的轨迹去和一整段真实的比，好让被优化的就是你真正要跑的那个。Marc'Aurelio Ranzato 和他在 Facebook 的同事同样在 2015 年提出了这个主张，论文是 *Sequence Level Training with Recurrent Neural Networks*。2015 年这两篇论文可以当一对来读：一篇修的是输入，另一篇修的是损失。

别推太远。让推演保持很短，并且从真实状态出发。它便宜又有效，代价是放弃了当初最吸引人的那种长长的想象未来。Michael Janner 和他的同事在 2019 年把它做成了一套方法，标题本身就是那个问题： *When to Trust Your Model*。

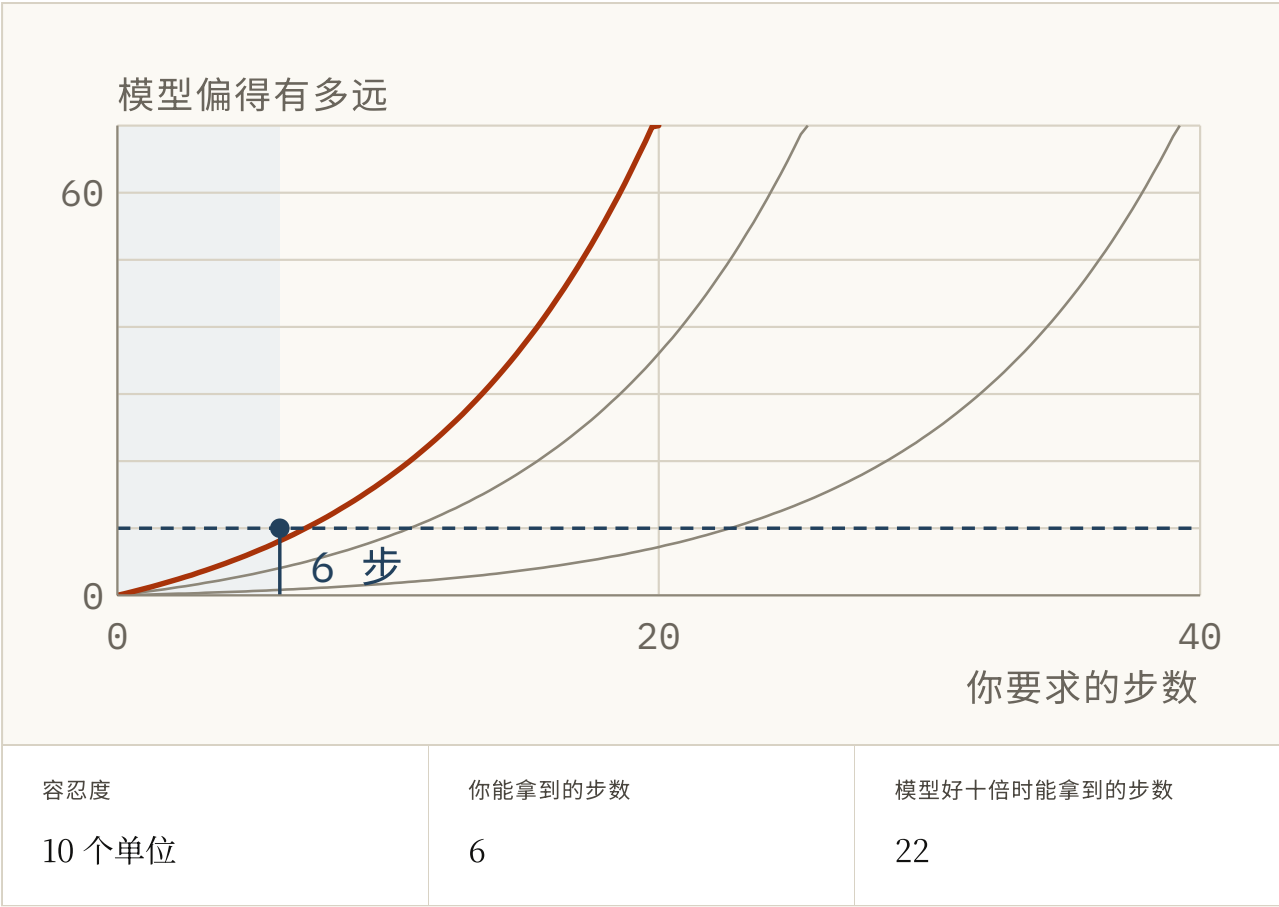


FIG. 5.8 能推多远是一个要你去测的数字，那就在这儿测一下。拖动「你愿意错到什么程度」，读出在模型的误差越过那条线之前，你能拿到多少步。然后换上一个好十倍的模型，看那个时程只往前挪了几步，而不是变成十倍。误差曲线仅作示意。

再看一眼。预测几步，执行其中一步，重新取一次观测，然后从头再来。机器人学里大半都是这个，也是这里最老的一个想法。一次测量能纠正状态里你看得见的那部分，但看不见的那部分里的误差，以及模型自身的偏差，都不会被清零。

它最老的形式是 Rudolf Kalman 在 1960 年代提出的滤波器，也就是每一台 GPS 接收机里跑的那套数学。那个滤波器就是这个循环，而它内部那次混合，是「先预测再纠正」最干净的一幅图，所以我们绕过去看一眼。想象一部雷达在跟踪一架飞机。在每一个时刻，关于飞机在哪儿，你手上都有两套说法。

一套来自物理：拿上一次的位置和上一次的速度，推算飞机现在应该在哪儿。另一套来自雷达，它给你一个回波。两套说法都不确定，各自其实都是一条钟形曲线，也就是高斯分布：最可能的位置在中间，两边越往外越淡。

把两条曲线乘起来，你会得到第三条钟形曲线，它落在两者之间，而且比两者都更窄。这就是新的信念，下面这幅图让你看着它成形。

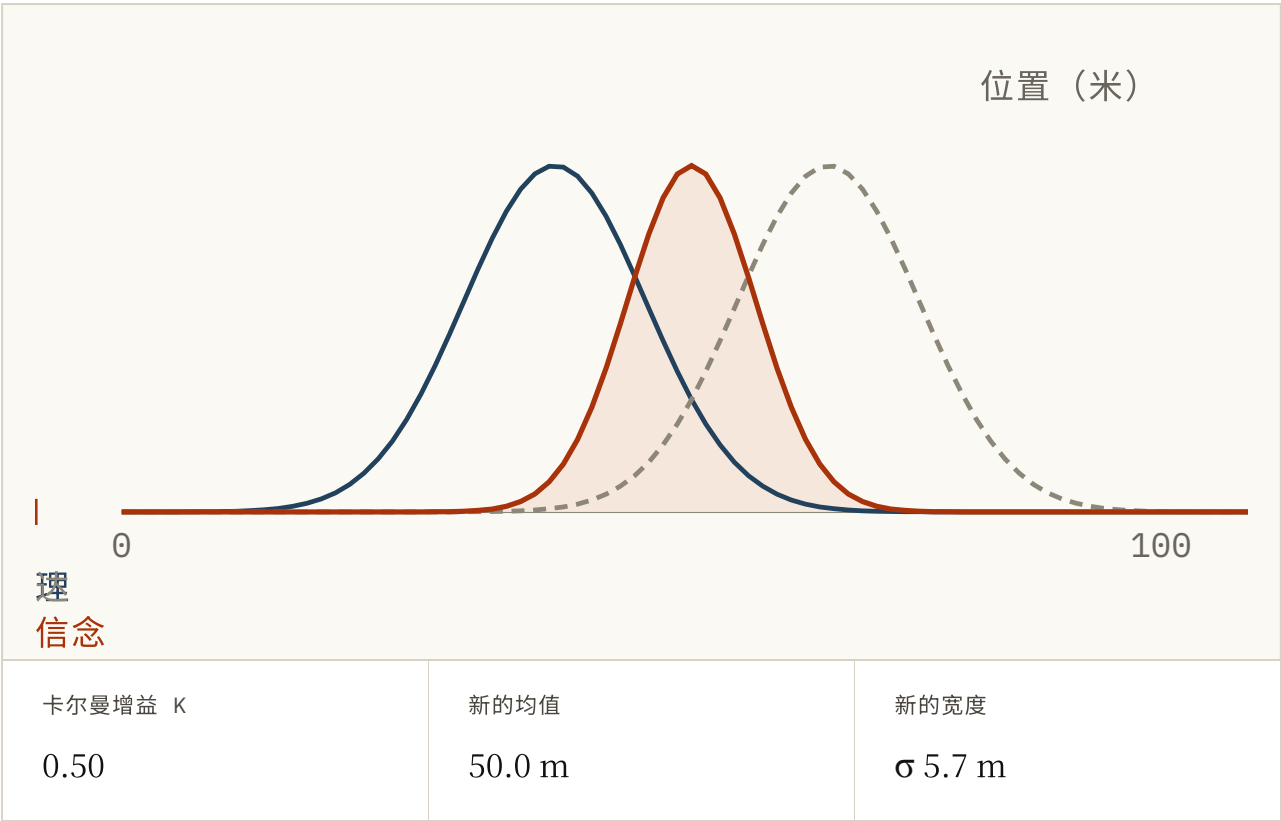


FIG. 5.9 这里是关于飞机在哪儿的两套说法，画成同一根轴上的两条钟形曲线。拖动「物理有多确定」和「雷达噪声」，看新的信念怎么往更尖的那套说法倾斜，同时又比两者都窄。卡尔曼增益 K 那个读数，说的就是它倾斜了多少。然后按一下「修正」，把这个新信念当成下一次物理预测，看着它随飞机往前走而重新变宽。位置仅作示意。

新信念从物理往雷达那边倾斜多少，由一个介于零和一之间的数字决定。这个领域把它叫做卡尔曼增益。从图里就能看出来：雷达越吵，增益越被拉向零，滤波器就守着物理；雷达越准，增益越被拉向一，滤波器就相信读数。

然后它一直重复：预测、纠正、预测、纠正。每一个新回波，不管它自己错得多离谱，都会让滤波器知道的东西更紧一点。哪怕它拿到过的每一个读数都是偏的，估计值依然紧贴着真相，而且它是一格一格做到这件事的，从来不用等未来。这就是「再看一眼」这条停战最原始的样子。它能成立，只是因为新的读数一直在来，而就算读数一直在来，它拉回来的也只是读数看得见的那一部分。



FIG. 5.10 同一次混合，反复地做，而状态的两半被分开画着。按一下「再看一眼」，盯住那条锯齿：每一次预测都把两边的误差往上推，每一个回波又把看得见的那一半直接拉回来。现在加上一阵滤波器从来没被告知过的风，继续按。看得见的那一半照样回得来，而看不见的那一半会落在一条它再也离不开的底线上。距离仅作参考。

这些都不算修好了，而把其中任何一种当成「修好了」来卖的论文，都是在夸大。每一种办法都承认，学出来的转移模型只在有限的时程内值得信任。工程的一部分，就是把那个时程测出来。

恢复是另一项本领

两个模型可以在通常的单步测试上打平，真跑起来却走向相反的结果。那个测试从来没有问过是什么把它们分开的。如果它们都只见过演示轨迹，那么它们的分数里没有任何东西能告诉你，稍微走偏一点之后它们会往哪边动。

在受扰动或自己生成的状态上训练，才算问到了这道题，而答案是另一项本领。拿下面这幅图试试。

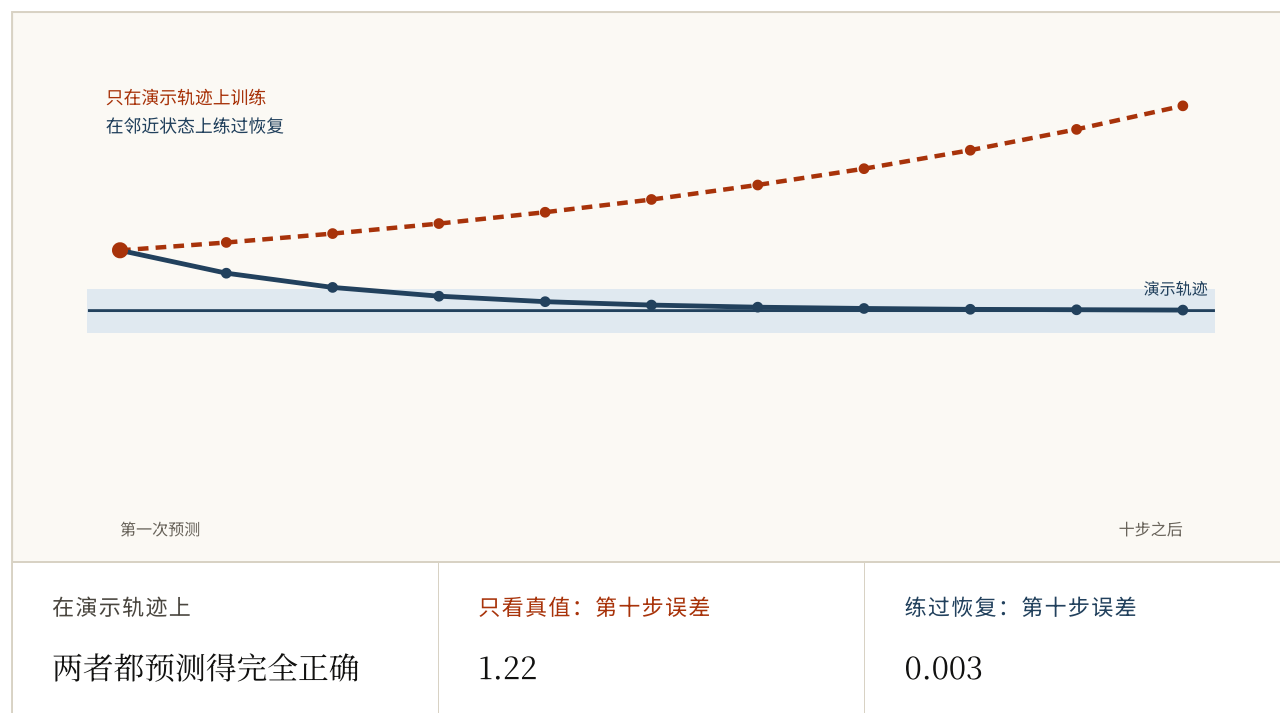


FIG. 5.11 两个玩具模型在演示轨迹上都完全正确。拖动起始偏差，让它们从轨迹外面出发。一个只学过线上的「下一步」，于是越走越远；另一个练过附近的状态，会弯回来。算术是编的，但这个区别是真的，它也是 Professor Forcing 背后的那个发现，我们接下来就讲。

计划采样会让学习者见到自己的状态，但它什么都不保证，因为这些状态会随着模型学习而不断变化。

一年之后是 Professor Forcing，论文题为 *Professor Forcing: A New Algorithm for Training Recurrent Networks*。它出自 Alex Lamb 和他在蒙特利尔、Yoshua Bengio 实验室的同事。两位 Bengio 是兄弟，而针对这道缺口最有名的两次进攻，一人贡献了一次。

它从另一侧攻这道缺口，输入原封不动。它做的是把模型训练到：不管是被教师强制还是自由运行，它的内部状态看上去都一样。裁判是第二个网络，专门被训练来分辨这两种跑法。模型必须骗过它，而这正是生成对抗网络背后的把戏：一个网络造假，另一个网络抓假。

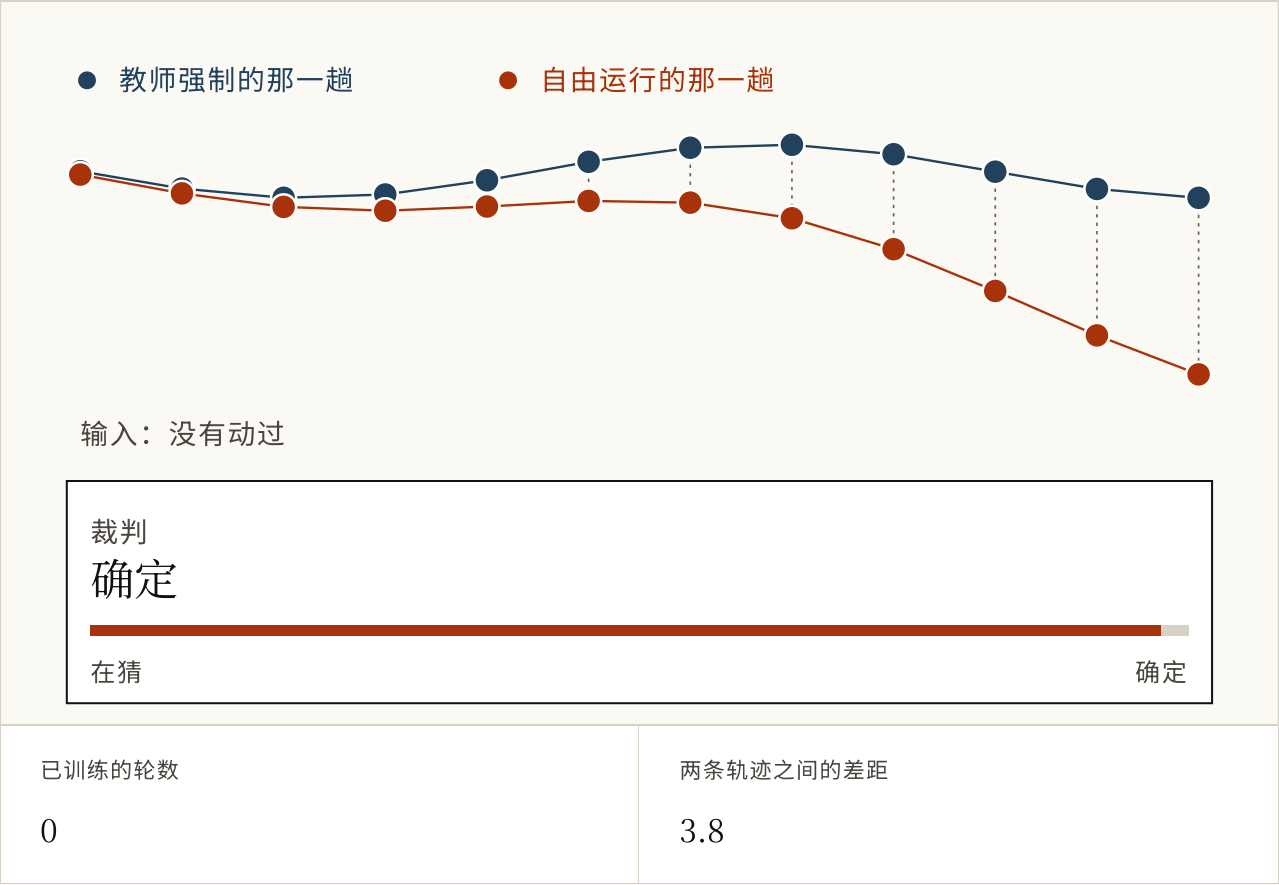


FIG. 5.12 同一个模型的两次运行，一次被教师强制，一次自由运行，各自留下一串隐藏状态的轨迹。右边的裁判被训练来指出哪串是哪串。按几次「训练一轮」，看自由那串怎么被拽向被强制的那串，直到裁判只能瞎猜。这里没有任何东西动过输入。轨迹仅作示意。

我觉得这两篇里它是被读得更少的那一篇，也许是因为 transformer 第二年就出现了，把整个领域的注意力都拿走了。而这个问题搬进了世界模型，在那里每一次推演都是一次自由运行。

这两篇论文的共同点是：恢复能力必须被放进训练目标里，因为模型不会自己学会它。

这一章里的一切都还拴在录下来的经验上：模型在它上面训练，对着它检验，也被它纠正。下一章要问的是，如果换成让智能体在模型里面学习，会发生什么。

希望这一章帮到了你。下面有一个简短的小测验，你可以拿它检查一下这些想法有没有留住。如果我哪里写错了，或者你知道比我用的更好的来源，「关于」页面上有一个更正的板块。我很乐意收到你的来信。

试一试

1 一个转移模型报告说自己的平均单步误差非常低。为什么这个数字并不能告诉你你知道的事？

- a. 它多半量错了
- b. 你会一次用它很多步，而从第一步之后，喂给它的就是它自己的答案，不是真相
- c. 单步误差总是被低估
- d. 平均值掩盖了异常值

答案 b. 你会一次用它很多步，而从第一步之后，喂给它的就是它自己的答案，不是真相. 这个测量是诚实的。它测的是一份这个模型永远不会被要求做的工作。

2 图里那条被纠正的线和那条自己跑的线，来自同一个模型、同一个每步偏差。为什么最后差这么远？

- a. 被纠正的那条用的是另一个模型
- b. 两次运行的随机噪声不同
- c. 自己跑的那条积累了更大的偏差
- d. 其中一条在每一步都被给了真实的上一个状态，所以它的错误从来没机会去喂任何东西

答案 d. 其中一条在每一步都被给了真实的上一个状态，所以它的错误从来没机会去喂任何东西. 纠正会在每一步把输入重置回真相，于是误差没法复利。两条线之间，模型本身什么都没变。

3 什么是教师强制？

- a. 训练时每一步都把真实的上一个值展示给模型，因为记录里装的就是这个
- b. 强制模型使用固定学习率
- c. 只在最难的样本上训练
- d. 在更大的模型标注的数据上训练

答案 a. 训练时每一步都把真实的上一个值展示给模型，因为记录里装的就是这个. 它让训练更稳，同时也意味着模型一次都没有被要求过从「有点偏」里恢复，而那恰恰是它之后唯一要做的事。

4 一段序列已经长过 transformer 的上下文窗口。哪句话准确？

- a. 序列变长只影响准确率，不影响计算量
- b. 注意力仍能逐字读取所有早先步骤
- c. 模型必须在窗口之外丢弃、压缩或检索；注意力只是推迟了这次决定
- d. 只有循环模型的记忆是有限的

答案 c. 模型必须在窗口之外丢弃、压缩或检索；注意力只是推迟了这次决定. 注意力不必把过去挤进一个固定状态，但这只在有限上下文之内成立。为当前预测做的加权读取本身也在压缩可见步骤。

5 对于一个短序列来说，哪一种每步更便宜？

- a. 永远是注意力
- b. 注意力，直到序列长到越过交叉点为止
- c. 两者一样
- d. 永远是摘要

答案 b. 注意力，直到序列长到越过交叉点为止. 更新一份摘要的开销不随长度变化，所以它只有在序列够长时才占优。在交叉点以下，全都看一遍是更便宜的那个选项。

6 为什么要在状态里同时带一个确定的部分和一个随机的部分？

- a. 为了让模型可导
- b. 为了支持更大的批量
- c. 为了用更多参数
- d. 只有确定的部分表示不了真正的不确定性；只有随机的部分则很难记住东西，因为每一步都在注入噪声

答案 d. 只有确定的部分表示不了真正的不确定性；只有随机的部分则很难记住东西，因为每一步都在注入噪声. 单独留任何一个都会失败，而且失败的方向不同。球的运动属于前者；门会不会开属于后者。

7 两个模型在演示轨迹上的单步误差相同。受到一点扰动后，只有一个会回来。原来的测试漏掉了什么？

- a. 模型自己的误差或外界扰动产生的状态上的恢复行为
- b. 摄像机分辨率
- c. 转移是不是确定性的
- d. 模型的参数量

答案 a. 模型自己的误差或外界扰动产生的状态上的恢复行为. 只沿中心线做单步测试，永远不会访问上线后一次小错会产生状态。恢复是另一种行为，必须在那条线之外训练或检验。

8 计划采样、整段推演训练、短时程、重新规划，这四者的共同点是什么？

- a. 它们修好了这个错配
- b. 它们都让模型每一步更准
- c. 没有一个消除了这个错配；每一个都只是限制它能造成多大破坏
- d. 它们只适用于循环模型

答案 c. 没有一个消除了这个错配；每一个都只是限制它能造成多大破坏. 诚实的总结是：一个学出来的转移模型只在某个时程内值得信任，而工程的一部分就是知道那个时程有多长。

FIG. 5.13 八道关于这一步和第二步问题的题。前三道会改变你读一篇论文头条数字的方式，所以后面的就算跳过，也试试这三道。

参考资料

Professor Forcing: A New Algorithm for Training Recurrent Networks

LAMB ET AL., 2016 ↗

不直接安排模型吃多少自己的输出，而是让教师强制和自由运行时的隐藏轨迹彼此相像。恢复能力必须进入训练目标。

<https://arxiv.org/abs/1610.09038>

Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks

BENGIO ET AL., 2015 ↗

把这个错配点了名，并正面处理：训练时就让模型吃自己的预测，而且随着它变好逐步加量。

<https://arxiv.org/abs/1506.03099>

Sequence Level Training with Recurrent Neural Networks RANZATO ET AL., 2015

↗

给整段推演打分，而不是给单步打分，好让被优化的东西就是你真正要跑的那个东西。

<https://arxiv.org/abs/1511.06732>

Learning Latent Dynamics for Planning from Pixels (PlaNet)

HAFNER ET AL., 2018 ↗

从图像里学出随机潜在动力学，然后在决策时对它做搜索。图 2.4 的真实版本。

<https://arxiv.org/abs/1811.04551>

Dream to Control (Dreamer) HAFNER ET AL., 2019 ↗

在想象出来的潜在轨迹上训练 actor 和 critic，于是动手的那一刻不必再跑昂贵的搜索。

<https://arxiv.org/abs/1912.01603>

Long Short-Term Memory HOCHREITER & SCHMIDHUBER, 1997 ↗

那份固定摘要，被做成能比梯度愿意的更久地抓住一些东西。需付费。

<https://doi.org/10.1162/neco.1997.9.8.1735>

Attention Is All You Need VASWANI ET AL., 2017 ↗

另一个答案：别再做摘要了，把每一步都留着，代价是开销随长度增长。

<https://arxiv.org/abs/1706.03762>

Efficiently Modeling Long Sequences with Structured State Spaces (S4)

GU ET AL., 2021 ↗

「做摘要」这条路带着更好的机器回来了，也是状态空间模型重新进入讨论的原因。

<https://arxiv.org/abs/2111.00396>

Mamba: Linear-Time Sequence Modeling with Selective State Spaces

GU & DAO, 2023 ↗

一份会根据自己正在看的东西来决定留什么的摘要，而这正是固定版本做不到的那个让步。

<https://arxiv.org/abs/2312.00752>

FIG. 5.14 排序是给要在这上面接着做事的人用的，而不是为了历史完整性，而且我全程优先选用了第一手材料。